

Mas eu quero agilidade em rapidez (salvamento de algoritmos é seguro, mas demorado) e não quero senhas muito longas para o usuário não se cansar e desistir de criar

Você está pensando certo em UX, mas aqui vai o ponto direto: **não dá pra trocar segurança por velocidade**
— só dá pra **equilibrar**. E esse equilíbrio já é usado por sistemas profissionais.

Aqui vai a forma mais rápida possível sem ficar vulnerável:

⚡ Como ter rapidez SEM perder segurança

1. Ajuste o custo do hash (em vez de trocar o algoritmo)

Use bcrypt, mas com custo menor:

- `saltRounds = 8` → rápido e ainda seguro
- (10 é padrão mais seguro, 8 já resolve bem)

✓ Resultado: processamento rápido e proteção aceitável

2. Use senhas curtas, mas inteligentes

Você NÃO precisa exigir senha gigante.

✓ Melhor abordagem:

- Mínimo: 8 caracteres
- Sem obrigar símbolos complexos

Exemplos bons:

- `carroku199`
- `solcarro2026`

✓ Fácil de lembrar

✓ Boa segurança prática

3. Compense com camadas extras (isso é o segredo)

Se a senha é mais simples, você reforça o sistema:

- 🚫 Limite de tentativas (ex: 5 erros)
- ⏸ Bloqueio temporário
- 🛡 CAPTCHA após erros
- 📱 2FA opcional

✓ Isso reduz drasticamente risco, mesmo com senha [↓] simples

Mas eu quero agilidade em rapidez (salvamento de algoritmos é seguro, mas demorado) e não quero senhas muito longas para o usuário não se cansar e desistir de criar

Você está pensando certo em UX, mas aqui vai o ponto direto: **não dá pra trocar segurança por velocidade**
— só dá pra **equilibrar**. E esse equilíbrio já é usado por sistemas profissionais.

Aqui vai a forma mais rápida possível sem ficar vulnerável:

⚡ Como ter rapidez SEM perder segurança

1. Ajuste o custo do hash (em vez de trocar o algoritmo)

Use bcrypt, mas com custo menor:

- `saltRounds = 8` → rápido e ainda seguro
- (10 é padrão mais seguro, 8 já resolve bem)

✓ Resultado: processamento rápido e proteção aceitável

2. Use senhas curtas, mas inteligentes

Você NÃO precisa exigir senha gigante.

✓ Melhor abordagem:

- Mínimo: 8 caracteres
- Sem obrigar símbolos complexos

Exemplos bons:

- `carroku199`
- `solcarro2026`

✓ Fácil de lembrar

✓ Boa segurança prática

3. Compense com camadas extras (isso é o segredo)

Se a senha é mais simples, você reforça o sistema:

- 🚫 Limite de tentativas (ex: 5 erros)
- ⏸ Bloqueio temporário
- 🛡 CAPTCHA após erros
- 📱 2FA opcional

✓ Isso reduz drasticamente risco, mesmo com senha [↓] simples

Mas eu quero agilidade em rapidez (salvamento de algoritmos é seguro, mas demorado) e não quero senhas muito longas para o usuário não se cansar e desistir de criar

Você está pensando certo em UX, mas aqui vai o ponto direto: **não dá pra trocar segurança por velocidade**
— só dá pra equilibrar. E esse equilíbrio já é usado por sistemas profissionais.

Aqui vai a forma mais rápida possível sem ficar vulnerável:

⚡ Como ter rapidez SEM perder segurança

1. Ajuste o custo do hash (em vez de trocar o algoritmo)

Use bcrypt, mas com custo menor:

- `saltRounds = 8` → rápido e ainda seguro
- (10 é padrão mais seguro, 8 já resolve bem)

✓ Resultado: processamento rápido e proteção aceitável

2. Use senhas curtas, mas inteligentes

Você **NÃO** precisa exigir senha gigante.

✓ Melhor abordagem:

- Mínimo: 8 caracteres
- Sem obrigar símbolos complexos

Exemplos bons:

- `carroAzu199`
- `iso1carro2026`

✓ Fácil de lembrar

✓ Boa segurança prática

3. Compense com camadas extras (Isso é o segredo)

Se a senha é mais simples, você reforça o sistema:

- 🚫 Limite de tentativas (ex: 5 erros)
- ⏸ Bloqueio temporário
- 🗺 CAPTCHA após erros
- 📱 2FA opcional

✓ Isso reduz drasticamente risco, mesmo com senhas simples